

# MC521 - Relatório VII

Júlio da Silva Telles

22 de Junho de 2026

## 1 A - Vacation (AtCoder - dp\_c)

O problema consiste em encontrar a máxima pontuação de felicidade acumulada que pode ser obtida ao longo de  $N$  dias de férias, sabendo que a cada dia é possível escolher uma entre três atividades alternativas ( $A$ ,  $B$  ou  $C$ ). A única restrição imposta é que não é permitido realizar a mesma atividade em dois dias consecutivos.

### 1.1 Solução

Podemos resolver este problema utilizando Programação Dinâmica (DP) em uma abordagem *Top-Down* com *Memoization*, garantindo uma complexidade de tempo linear de  $\mathcal{O}(N)$ . O algoritmo segue os seguintes passos estruturados:

1. **Base da Recursão:** Quando o índice do dia atinge o limite final estipulado ( $i = n$ ), a função retorna diretamente o peso bruto associado àquela escolha específica ( $w[i][j]$ ), servindo como ponto de parada do empilhamento recursivo.
2. **Memoização do Estado:** Armazenamos os resultados intermediários calculados em uma matriz bidimensional de soluções (`sol`). Se o estado correspondente ao dia  $i$  e à atividade  $j$  já foi computado previamente (`s[i][j] != 0`), a resposta guardada é retornada instantaneamente, mitigando a redundância de recálculos em subproblemas coincidentes.
3. **Transição entre Estados:** O valor ótimo para o estado corrente `s[i][j]` é definido pela pontuação do dia atual adicionada ao maior valor retornado pelas opções válidas do dia subsequente. Para evitar repetir a mesma atividade, emprega-se aritmética modular para computar os índices das outras duas ações disponíveis:  $(j + 1) \% 3$  e  $(j + 2) \% 3$ .

### 1.2 Código Fonte

```
1 int dp(int i, int j, int n, vector<vi> &w, vector<vi> &s){
2     if (i == n) return w[i][j];
3     if (s[i][j]) return s[i][j];
4     s[i][j] = w[i][j] + max(dp(i + 1, ((j + 1) % 3), n, w, s), dp
5         (i + 1, ((j + 2) % 3), n, w, s));
6     return s[i][j];
7 }
```

## 2 E - Maximum Subarray Sum (CSES - 1643)

O problema consiste em encontrar a maior soma possível de um subarranjo contíguo e não vazio contido dentro de um vetor unidimensional contendo  $N$  números inteiros (que podem ser positivos ou negativos).

### 2.1 Solução

Podemos modelar e resolver este problema utilizando Programação Dinâmica linear, baseando-se no conceito fundamental do consagrado Algoritmo de Kadane. Essa modelagem nos provê uma complexidade de tempo altamente eficiente de  $\mathcal{O}(N)$  e complexidade de espaço de  $\mathcal{O}(N)$ . O fluxo de execução obedece aos seguintes passos:

1. **Definição do Estado da DP:** Criamos um vetor DP de tamanho  $N + 1$ , inicializando a base  $DP[0] = 0$ . Cada posição  $DP[i+1]$  armazena a maior soma terminando obrigatoriamente no  $i$ -ésimo elemento analisado.
2. **Transição e Acúmulo:** Ao iterar pelo vetor de entrada, a equação de recorrência avalia se é mais vantajoso estender o subarranjo acumulado anteriormente ou "reiniciar" a contagem a partir de zero caso a soma se torne prejudicial (negativa):  $DP[i + 1] = \max(0, DP[i] + w[i])$ . A variável global `maxsum` monitora e retém a crista máxima histórica registrada pelo acumulador ao longo da varredura.
3. **Tratamento de Vetores Estritamente Negativos:** Caso todas as entradas do vetor sejam estritamente negativas, o algoritmo por padrão zeraria o acumulador. Para sanar essa exceção e obedecer à restrição de subarranjos não vazios, controlamos o maior elemento isolado do arranjo através da variável `maxi`. Se `maxi <= 0`, a resposta final ótima será unicamente esse maior elemento isolado.

### 2.2 Código Fonte

```
1 int main(){
2     ll n, maxi = -1e10;
3     cin >> n;
4     vll DP(n + 1, -1e9), w(n);
5     for (int i = 0; i < n; i++)
6         cin >> w[i];
7     DP[0] = 0;
8     ll maxsum = DP[0];
9     for (int i = 0; i < n; i++){
10         if (w[i] > maxi)
11             maxi = w[i];
12         DP[i + 1] = max(0LL, DP[i] + w[i]);
13         maxsum = max(maxsum, DP[i + 1]);
14     }
15     if (maxi > 0)
16         cout << maxsum << '\n';
17     else
18         cout << maxi;
19 }
```